FINAL PROJECT REPORT

Static versus Dynamic Task Scheduling on a Hybrid CPU/GPU Machine

An AI-assisted heterogeneous computing framework: survey, base-work redo, gaps, implementation and measured evaluation

Course: Computer Architecture and Organization

Submitted by

Lakshay S (Reg. No. 26MAI0064)

Arkkadu Bharath Reddy (Reg. No. 26MAI0058)

Guide: Dr. Yokesh Babu S, Associate Professor Grade 1

Institution / Department: [to be filled by the authors]

Contact e-mail: [to be filled by the authors]

Date of this version: 3 November 2026

This one document is the consolidated upload for DA1 (survey), DA2 (base-work redo and gaps) and DA3 (proposed work).

Declaration

We declare that this report describes the work carried out for the project named on the title page, that all measured numbers in it are read from result files stored in the project repository, that nothing in it is simulated unless labelled *simulation* or *emulation*, and that statements about literature, patents and products are limited to what the cited documents state. Items that were not measured are listed as *not established* and are not presented as results. Signatures of the authors and the guide are to be added before submission.

Table 1. Evidence labels used throughout

Label	Meaning
real	executed on the project laptop (Intel Core i5-13500HX, RTX 3050 6 GB, Windows 11); outputs verified against a CPU reference
emulation	remote worker replaced by a local process behind a loopback socket and a delay shim; never presented as a real network or device
plan	written down but not run (the Raspberry Pi validation)
confirmatory / exploratory	confirmatory: the ten contrasts of the primary final session, fixed in the protocol and Holm-adjusted; exploratory: everything else, unadjusted

Contents

1. Introduction and Scope 3
2. Domain and Problem Substantiation 4
3. Survey: Publications, Patents, Products and Trends 4
4. Redoing the Base Work and Identifying Gaps 6
5. System Design and Implementation 7
6. Methodology 10
7. Results 11
8. Existing versus Proposed Implementation 17
9. Testing, Reproducibility and Demonstration Guide 17
10. Publication and IPR Route 18
11. Conclusion, Limitations and Future Work 18
References 20
Appendix A. File Map 23

1. Introduction and Scope

Heterogeneous computing combines processing elements of different speed in one system: performance (P) and efficiency (E) cores, a GPU, a single-board computer such as a Raspberry Pi 5 [1], or a remote node. Applications are *workflows*: directed acyclic graphs (DAGs) of tasks joined by data dependencies, such as the Montage mosaic [2], [3] or image pipelines. A *scheduler* decides the order in which ready tasks are served and the worker each one uses; *offloading* is the case in which the worker is on another device.

The project asks a practical question: **does a dynamic scheduler, which decides while the workflow runs, finish sooner than a static plan computed beforehand from the same cost model, on a hybrid CPU+GPU machine, once planning and start-up time are counted, and under which conditions?** Static schedulers such as HEFT and PEFT [4], [5] fix every assignment in advance; dynamic ones, such as StarPU [6] and learned schedulers [7], [8], [9], decide at run time.

The deliverables of the course are covered in this report: domain and problem statistics (Chapter 2), surveys of publications, patents and products and technology trends (Chapter 3), redo of the base work and gap identification (Chapter 4), the proposed work with its architecture and methodology (Chapters 5 and 6), results (Chapter 7), the difference between the existing and the proposed implementation (Chapter 8), the demonstration guide and reproducibility (Chapter 9) and the publication and IPR route (Chapter 10). Scope: one laptop with six P cores, seven E cores and one CUDA worker (RTX 3050); the remote worker was emulated; **no Raspberry Pi or real-network result exists**.

1.1 Contributions

- A reproducible runtime and experiment framework that executes real task graphs on pinned P/E workers and a CUDA worker, verifies every output and logs every decision.
- A final benchmark of static plans (HEFT, PEFT, HEFT with end-to-end terms) against dynamic dispatch (blind B4, a parameter-aware rule, a predicted-rank dispatcher) with *one* cost model, end-to-end timing and per-task GPU eligibility taken from measurements.
- A pre-stated confirmatory family of ten contrasts, six workloads, and an honest account of limits: a simple rule gets almost all of the gain, there is no gain without cost spread, the GPU model has 23% to 31% error.

2. Domain and Problem Substantiation

Four trends make scheduling on heterogeneous resources important; each figure was checked against its source, and two are secondary reports.

Table 2. Verified statistics that motivate the problem

Indicator	Value	Source
Data-centre electricity	about 415 TWh in 2024 (about 1.5% of global use); about 945 TWh projected for 2030	[10]
Accelerated HPC systems	232 of 500 TOP500 systems (46.4%) use accelerators, June 2025	[11]
Connected IoT devices	18.5 billion at end-2024 (+12%); about 39 billion forecast for 2030	[12]
Under-used clusters	Alibaba cluster: more than 80% of the time at 10% to 30% CPU use	[13]
Hybrid client CPUs	Intel Thread Director on 12th to 14th generation Core processors	[14]

The amount of heterogeneous capacity is large and growing, and it is often wasted, so placement quality turns into energy, cost and latency. On the project machine a 512-pixel denoise task with 30 iterations takes 156 ms on a P core and 560 ms on an E core (3.6 times longer), and the GPU is up to 15 times faster for edge detection at 1024 pixels but only breaks even at small tiles because host-device copies dominate. A scheduler that ignores these ratios sends heavy work to slow cores.

3. Survey: Publications, Patents, Products and Trends

3.1 Publications

We analysed 12 supplied papers in full and one at abstract level. The 13 solutions fall into four families: learned task selection with a heuristic placement rule [7], [15], [9]; end-to-end deep reinforcement learning for offloading [16], [17], [18], [19], [20], [21], [8], [22], [23]; lightweight runtime scheduling [24]; and prediction-driven list scheduling [25], [6], [4], [5].

Table 3. Comparison of the main published solutions

Ref.	Method	Platform	Reported result	Limitation for our goal
[7]	GNN + RL selects the task, DEFT places it	simulated cluster	up to 26.7% shorter makespan	simulation; no variance test
[24]	lightweight asynchronous scheduling	real Zynq CPU+FPGA	up to 2x; overhead measured	FPGA specific; no DAG, no prediction
[8]	dependency-aware RL, bandwidth	simulation + 3 Jetson boards	lower time and energy	network focus; DRL cost
[9]	GCN-Transformer + PPO	simulated multicore	11.09% lower time and power	simulation; heavy model
[22]	meta-RL + seq2seq	simulated edge	21% less delay, 19% less energy	simulation

Ref.	Method	Platform	Reported result	Limitation for our goal
[25]	runtime prediction drives HEFT	simulation	close to perfect-knowledge HEFT	no real execution

Three findings structure the work. Only two of 13 papers use physical hardware; no paper reports a confidence interval; and none compares against an *informed* simple rule or gives a static and a dynamic scheduler the same cost model and times both end to end. These are the gaps G1 to G10 of Chapter 4. The 13 papers are the supplied base set, not a systematic review, so we write "not found in the sources inspected".

3.2 Patents (IPR)

The base patent is US 11,561,826 B1 (Xilinx; granted 24 Jan. 2023) [26]. Its independent claims recite kernel objects tied to compute circuits, a task graph, per-kernel task queues and thread-based enqueue and dequeue; dependent claims add in-degree counters (claims 9 and 10) and performance-based thread adjustment (claim 8). No claim recites prediction of execution time, a transfer-cost model or offloading to a remote device.

Table 4. Comparative analysis of patents (legal status is a Google Patents assumption, not an opinion)

Patent	Discloses	Prediction	Relation to this work
[26] Xilinx, US 11,561,826 B1	task graph, per-kernel queues, in-degree gating	no	base work; mapped to blind dispatch B4
[27] IBM, US 11,740,933 B2	learning agent ranks DAG tasks; rank-ordered dispatch	learned rank	closest; ranking is not new
[28] IBM, US 11,966,776 B2	learned DAG policy with execution estimates	yes	deadline oriented
[29] Intel, US 10,186,007 B2	CPU+GPU online profiling and work split	online	no DAG, no learned model
[30] Samsung, US 10,162,671 B2	duration prediction on heterogeneous cores (lapsed)	yes	contention-aware

Task graphs, queues and gating are claimed by the base patent; rank-ordered dispatch with a learning agent is disclosed by IBM. The project therefore treats B4 as a comparison baseline and positions its contribution as a measured study, not as a new patentable algorithm; a professional prior-art search is required before any filing.

3.3 Company products

Table 5. Products that decide where work runs (only what the cited pages state)

Product	Scope	Heterogeneity handled	Prediction / learning
Intel Thread Director [14]	client CPU	P versus E cores, hardware feedback	hardware classification
Windows QoS [31]; Linux EAS [32]	OS scheduler	QoS levels; asymmetric capacity and energy model	heuristics / energy model

Product	Scope	Heterogeneity handled	Prediction / learning
Kubernetes [33], AWS Batch [34], Slurm [35], Ray [36]	cluster, batch	declared CPU/GPU resources	none stated
StarPU [6]	task runtime	CPU/GPU tasks, data transfers	performance models

Operating-system features place threads, cluster products place containers or jobs against declared requests, and only workflow runtimes such as StarPU understand task graphs. No page we read combines DAG knowledge, learned per-resource estimates and published decision-overhead measurements on a laptop-class CPU/GPU system.

3.4 Technology trends: introduction and adoption status

Table 6. Trends, their state and relevance

Trend	State (as documented)	Relevance
Hybrid P/E cores	mainstream in client CPUs [14]	core type changes task time by 2.8 to 3.6 times here
On-device GPU / NPU	NPU of 40 TOPS or more required for Copilot+ PCs [37]	the CUDA GPU is used as a worker
Learned DAG scheduling	research; simulation in most papers	tested here with a much cheaper model
LLM-agent schedulers	2025 to 2026 preprints [38], [39]	noted as a direction, not evaluated
Runtime performance models	established in StarPU [6]	same idea with overhead accounting
Edge boards as remote workers	Raspberry Pi 5 [1]	emulated only; no real-link result

4. Redoing the Base Work and Identifying Gaps

A laptop cannot reproduce the FPGA system of the base patent, so what can be redone is its *scheduling idea*: a worker takes a task only if the task is eligible for it, ready when all parents completed, with no estimate of execution time. We map it to policy B4 (first idle eligible worker, priority P then E) and reimplement HEFT and PEFT [4], [5]; the repository tests reproduce the published worked examples.

Table 7. Redo of the base idea on real executions (paired makespan change, negative = shorter; earlier study)

Workload	Comparison	Result	Reading
Montage, 59 tasks (9 blocks)	B4 vs static P-only	+47.87% [+37.5, +54.6]	greedy use of slow E cores hurts
Uniform batch + GPU (30)	static P-only vs B4	+35.9%	B4 keeps all workers busy
Hidden cost, 16 heavy (36)	hand rule vs B4	−19.85%	B4 is blind to cost
Hidden cost, 4 heavy (12)	hand rule vs B4	−26.94%	B4 worse than P-only

B4 is a good way to keep workers busy and a poor one when task cost differs and is invisible to the scheduler: in 93.6% of 20,046 stored B4 dispatches only one eligible worker was idle, so

order, which B4 leaves to first-come first-served, is where the gain lies. Instrumenting B4 also exposed a defect: an idle CUDA worker that no task can use defeats the early exit of the dispatch loop, so 10,042 of 10,299 selection calls returned nothing; a decision-preserving fix moved the dispatcher's gain by 0.8 points only.

Table 8. Gaps identified and the project response

Gap	Evidence	Response
G1 real hardware	2 of 13 papers use devices	P/E pools, CUDA worker, real workloads
G2 scheduler overhead	decision time in 4 of 13 papers	per-dispatch and end-to-end timing logged
G3 statistics	no confidence intervals	paired bootstrap, Holm, frozen protocol
G4 informed baselines	no informed rule in any paper	hand rule, HEFT, PEFT, HEFT-E2E
G5 cost-blind base idea	B4 5% to 37% slower than a rule	predicted-rank dispatcher
G6 accuracy is not value	accurate remote predictions, no gain	reported separately
G7 generalisation	one instance per setting	six workloads, 20 unseen orders earlier
G8 remote reality	simulated networks	emulation labelled; Pi not measured
G9 / G10 static vs dynamic	HEFT baseline only in simulation	final benchmark, one model, end-to-end timing, CPU and GPU

5. System Design and Implementation

The coordinator converts a workflow into a DAG, keeps the ready queue with dependency counters, and dispatches each task to a worker process pinned to one logical processor (six P workers, seven E workers, one CUDA worker G); a reserved core runs the coordinator. Every policy honours per-task eligible pools through one helper, every output is compared with a CPU reference, and an event log records each dispatch with its decision time. One cost model and one GPU-eligibility table feed both the static planners and the dynamic dispatchers (Figure 1).

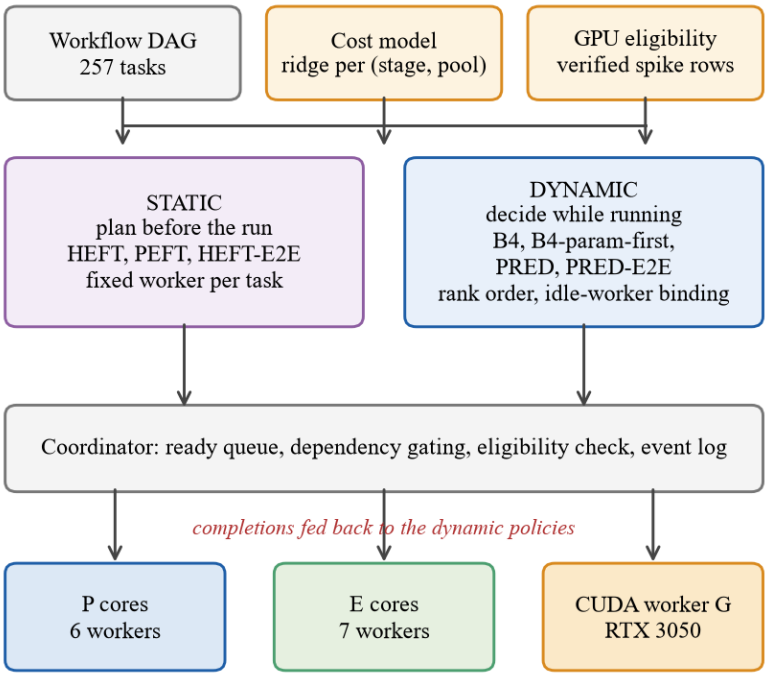


Figure 1. Framework of the final study: shared cost model and GPU eligibility; static and dynamic policy families; shared coordinator, event log and worker pools.

Table 9. Policies compared (all share eligibility rules and, where they predict, the same ridge cost model)

Policy	Kind	Rule
B4	dynamic	first idle eligible worker per ready task in FIFO order; no estimate
B4_param_first	dynamic	B4 with the queue sorted by the cost-relevant task parameter (a hand rule)
PRED_RANK_GREEDY	dynamic	order by predicted upward rank; bind only to an idle worker with the least predicted kernel time
PRED_RANK_GREEDY_E2E	dynamic	as PRED, minimising predicted occupancy: transfer + kernel + set-up + inter-process time
HEFT / PEFT / HEFT_E2E	static	plan before the run from the same model (HEFT_E2E adds the end-to-end terms); fixed worker per task
HEFT_oracle	static, reference	plan from class-median costs of the calibration sessions; not a bound

Cost model. A ridge regression per (stage, pool) on the numeric task parameters and size that vary in the logs and on their pairwise products; it names no stage or parameter, refuses an unseen stage or pool, and is refitted by one command. More elaborate predictors (constant, table, analytic, moving history, ridge with context, CatBoost, a small MLP, an oracle replay) were tried in earlier phases and did not improve scheduling enough to justify their cost. On hold-out tasks the median error is 4% to 11% for CPU stages and 23% to 31% for GPU stages.

Dispatch pass. Whenever a task becomes ready or a worker becomes idle the coordinator runs one pass, identical for all policies except for two hooks:

1. take the ready tasks and let the policy reorder them (the proposed policy sorts by predicted upward rank, rank(t) = predicted time(t) + the largest rank among the successors of t);
2. for each task in that order ask the policy for a worker; a late-binding policy may name only an idle eligible worker, otherwise the task stays queued;
3. check eligibility and idleness (a violation raises a contract error), bind the task and log the dispatch with its decision time;
4. stop when no worker is idle or all ready tasks were visited; on completion log the result and release the dependents.

Table 10. Event log and run manifest content

Record	Fields (selection)
task_ready, task_dispatched	task id, worker, pool, queue wait, worker load, decision time
task_submitted, task_completed	kernel time, start and end timestamps, transfer time, output size and SHA-256
worker_ready, run_started, run_finished	pid, requested and actual affinity; makespan, status, tasks per pool
run manifest	condition, policy, seed, warm-up flag, output verdict, event-log hash, git commit and dirty flag, source and design hashes

Table 11. Cost-model error on hold-out tasks (E = efficiency core, P = performance core, G = CUDA worker)

Stage on pool	n	MAE (ms)	Median abs. error (%)
aggregate on E	11	0.02	7.2
aggregate on P	13	0.01	7.6
denoise on E	641	19.89	4.0
denoise on G	51	100.40	22.8
denoise on P	844	17.73	10.9
edges on E	254	1.94	4.6
edges on G	607	4.23	30.4
edges on P	675	2.17	11.1
features on E	224	0.44	4.9
features on G	417	2.79	31.3
features on P	895	0.53	10.3
source on E	661	33.39	8.9
source on P	875	20.36	8.8

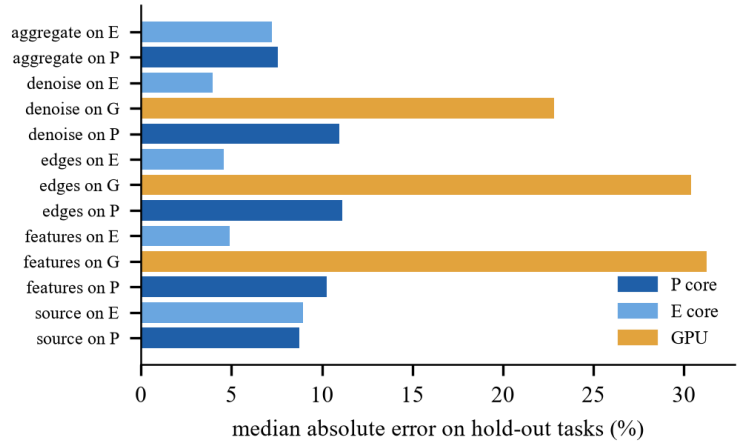


Figure 2. Median absolute error of the cost model on hold-out tasks, by stage and pool.

GPU eligibility. A stage may run on the CUDA worker only where a recorded spike measurement shows a verified speed-up at that size and iteration count (for example edges 7.68x, features 2.88x, denoise 1.20x to 1.77x at 15 to 120 iterations; denoise at 1 or 2 iterations is dropped). The configurations are *cpu* (P+E only), *gpu* (strict: original record) and *ext* (strict plus denoise at the extended record's iteration counts); none is GPU-only. **Timing.** The end-to-end makespan runs from the start of `run()` to the last completion; initialisation and policy set-up are logged separately, so a static plan's planning time is counted.

Code and tests. The runtime package has 107 files and 12,578 lines of Python, the scripts 9,709 lines and the tests 7,263 lines; 520 of 521 tests pass (the one failure is a test of the Montage profiler that refuses to start when free RAM is below its own 2 GB gate and is environmental). A local dashboard shows system state, run replays and the stored final results; two later versions, in separate packages, show the GPU with live load and run one live block of the final benchmark with any final policy.

6. Methodology

Machine and workloads. Intel Core i5-13500HX (6 P, 8 E cores), RTX 3050 6 GB, 15.7 GB RAM, native Windows 11. The image pipeline (source, denoise, edges, features, aggregate; 512 px, 64 chains, 257 tasks) has H heavy chains (denoise 120 iterations) among light ones (2 iterations). The primary workload h16_l120 has 16 heavy chains (24 included blocks, 576 included runs); five exploratory workloads use 4 or 40 heavy chains, cost ratios 20 and 7.5, and a uniform workload (13 blocks each).

Table 12. Evaluation workloads (runs per session; included runs in brackets; the first block of each session is a warm-up)

Workload	Heavy chains / iterations	Blocks	Runs
h16_l120 (primary)	16 heavy, 120 vs 2 iterations (ratio 60)	25	600 (576)
uniform	none; cpu and gpu only	13	208 (192)
h04_l120	4 heavy, 120 vs 2	13	312 (288)
h40_l120	40 heavy, 120 vs 2	13	312 (288)
h16_l40	16 heavy, 40 vs 2 (ratio 20)	13	312 (288)
h16_l15	16 heavy, 15 vs 2 (ratio 7.5)	13	312 (288)

Data separation and protocol. Calibration sessions fitted the model and the end-to-end terms, hold-out sessions checked them, and the evaluation sessions came afterwards with new chain orders; the protocol and design hashes were logged before the first evaluation session. Conditions run once per block in seeded random order; free memory must exceed 2.4 GB at the start and 0.8 GB per block and the laptop must be on mains power (thresholds were never lowered; aborted attempts are kept and never pooled).

Statistics. The effect of a condition against a reference is (exp(mean ln(M_c/M_r)) − 1) × 100% over paired blocks, with a percentile bootstrap interval (10,000 resamples) stratified by chain order, sign-flip *p*-values, Holm correction over the ten contrasts C1 to C10 of the primary session and a ±2% equivalence margin. Expected outcomes were written before the data; they held for C1, C2, C7, C9 and C10 and were wrong for C3 and C4.

7. Results

7.1 Primary workload and confirmatory contrasts

The first table below gives the end-to-end makespan (geometric mean, 24 blocks). In every hardware set the cost-aware dynamic policies are within about 1% of each other and clearly faster than blind B4; the static HEFT plan is slower than PRED and PEFT is the slowest static plan. All included runs had verified output.

Table 13. Primary workload h16_l120: end-to-end makespan in seconds

Policy	CPU only	Strict GPU	Extended GPU
B4 (blind, dynamic)	4.010	4.075	4.123
B4 param-first (rule, dynamic)	3.310	3.218	3.069
PRED (dynamic)	3.297	3.213	3.050
PRED-E2E (dynamic)	3.318	3.212	3.033
HEFT (static)	3.478	3.660	3.114

Policy	CPU only	Strict GPU	Extended GPU
HEFT-E2E (static)	3.499	3.512	3.173
PEFT (static)	4.093	4.050	3.689

Table 14. Confirmatory contrasts (Holm family of ten; negative = first policy faster)

Id	Contrast	Effect % [95% CI]	Holm p	Result
C1	PRED vs B4 (CPU)	−17.8 [−18.8, −16.7]	0.001	faster
C2	PRED-E2E vs PRED (CPU)	+0.6 [+0.1, +1.3]	0.141	equivalent (within ±2%)
C3	PRED vs HEFT (CPU)	−5.2 [−5.9, −4.6]	0.001	faster
C4	PRED-E2E vs HEFT-E2E (CPU)	−5.2 [−5.9, −4.5]	0.001	faster
C5	PRED vs HEFT (strict GPU)	−12.2 [−13.4, −11.1]	0.001	faster
C6	PRED-E2E vs HEFT-E2E (strict GPU)	−8.5 [−9.7, −7.6]	0.001	faster
C7	PRED-E2E vs PRED (strict GPU)	0.0 [−0.7, +0.5]	0.960	equivalent (within ±2%)
C8	PRED vs B4 (strict GPU)	−21.2 [−22.6, −19.7]	0.001	faster
C9	PRED, strict GPU vs CPU	−2.6 [−3.3, −1.7]	0.001	faster
C10	B4, strict GPU vs CPU	+1.6 [+0.2, +3.0]	0.141	inconclusive

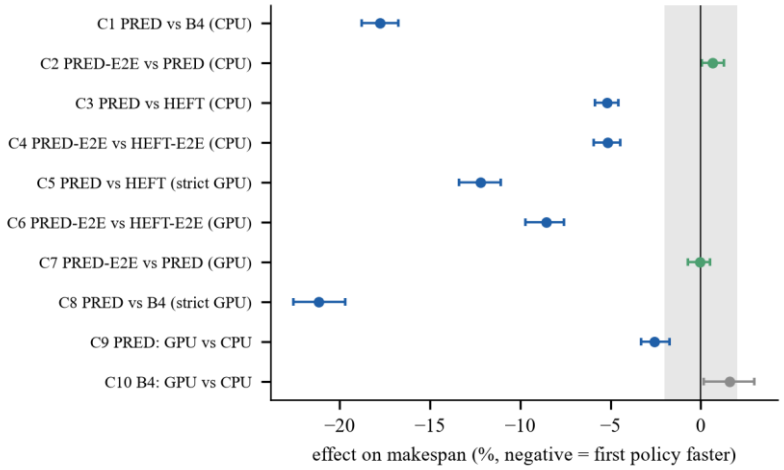


Figure 3. Confirmatory contrasts C1 to C10 with 95% intervals; the grey band is the ±2% margin.

The dynamic dispatcher beats blind B4 by 17.8% on the CPU (C1) and 21.2% with the strict GPU set (C8), and beats the static HEFT plan from the same model by 5.2% on the CPU (C3) and 12.2% with the strict GPU set (C5). End-to-end cost terms change nothing (C2, C7: equivalent within ±2%). The GPU helps the dispatcher by 2.6% (C9) but not blind B4 (C10: +1.6%, inconclusive); with the extended set the gain over the CPU is 7.5% [7.2, 7.8].

7.2 Six workloads, GPU and limits

Table 15. Effect of PRED (%, negative = PRED faster; exploratory except h16_l120)

Workload	vs HEFT, CPU	vs HEFT, strict GPU	vs B4, CPU	vs B4, strict GPU

Workload	vs HEFT, CPU	vs HEFT, strict GPU	vs B4, CPU	vs B4, strict GPU
h16_l120	−5.2 [−5.9, −4.6]	−12.2 [−13.3, −11.1]	−17.8 [−18.8, −16.7]	−21.2 [−22.5, −19.7]
uniform	−0.2 [−1.2, +0.9]	−3.7 [−4.7, −2.8]	+0.0 [−2.5, +2.3]	0.0 [−1.3, +1.1]
h04_l120	−2.4 [−3.9, −0.6]	−12.8 [−13.6, −12.2]	−39.4 [−42.3, −36.3]	−40.1 [−44.5, −35.0]
h40_l120	−4.8 [−5.4, −4.2]	−8.6 [−10.7, −6.6]	−2.8 [−3.4, −2.2]	−6.0 [−7.8, −4.2]
h16_l40	−3.2 [−3.9, −2.3]	−17.1 [−17.8, −16.4]	−4.8 [−7.3, −2.4]	−7.9 [−9.2, −6.4]
h16_l15	−1.4 [−1.9, −0.9]	−12.2 [−13.2, −11.1]	−0.7 [−1.5, +0.2]	+1.1 [+0.3, +1.9]

The advantage depends on how much task cost varies: it is largest when 4 of 64 chains are heavy (about 40% against B4), shrinks with cost ratios of 20 and 7.5 and with 40 heavy chains, and is zero on the uniform workload (+0.0% [−2.5, +2.3] on the CPU). With ratio 7.5 and the strict GPU set PRED is slightly slower than B4 (+1.1% [+0.3, +1.9]). Under the model-based policies the CUDA worker takes roughly 15% to 20% of the 257 tasks; blind B4 sends about 32% to the GPU with the strict set and is slower for it.

- **A simple rule gets almost all of the gain.** PRED against B4_param_first is −0.4% [−0.9, +0.1] (CPU), −0.2% [−0.9, +0.7] (strict GPU) and −0.6% [−1.1, −0.0] (extended GPU), exploratory.
- **A hybrid selector is not justified:** the maximum headroom in 17 development cells was 1.1% (mean 0.08%), none above the 2% margin.
- **The cost model is weakest on the GPU** (23% to 31% median error); part of the dynamic-versus-static gap on GPU sets may reflect model error and not only decision timing, which was not separated.
- **Timing convention:** the end-to-end and old makespan differ by 12.5 ms (B4) to 81.3 ms (PEFT) on 3 to 4 s runs; no conclusion flips.
- **Two sessions disagree:** an earlier session found HEFT level with the dispatcher (+1.10% [−0.53, +2.81]); the final session finds PRED about 5% faster. The difference was not isolated; both are reported.

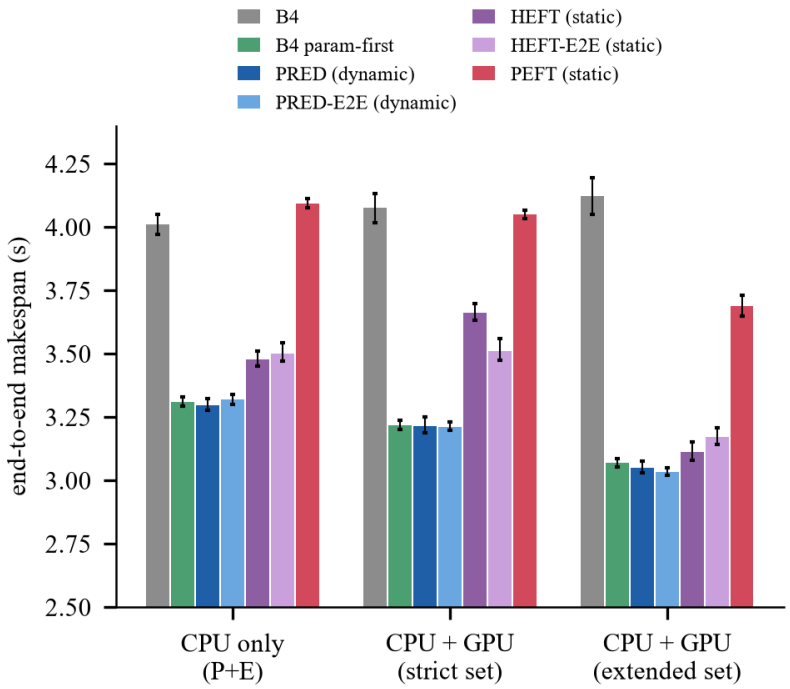


Figure 4. End-to-end makespan of each policy in the three hardware sets (primary workload; geometric mean, 95% interval).

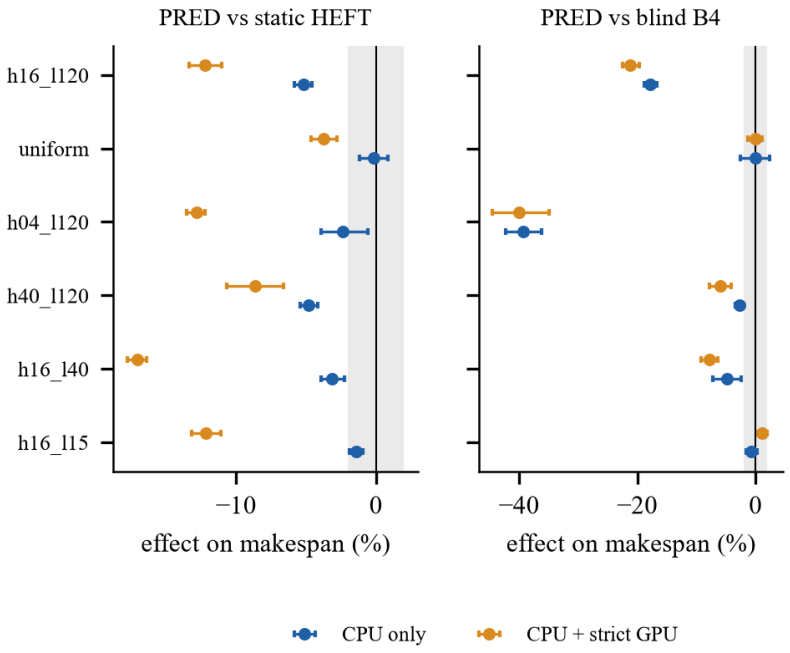


Figure 5. PRED against static HEFT (left) and blind B4 (right) on the six workloads, CPU only (blue) and strict GPU set (orange).

Table 16. Effect of the hardware set within each policy on the primary workload (%, negative = faster)

Policy	Strict vs CPU	Extended vs CPU	Extended vs strict

Policy	Strict vs CPU	Extended vs CPU	Extended vs strict
PRED (dynamic)	−2.6 [−3.3, −1.7]	−7.5 [−7.8, −7.2]	−5.1 [−5.7, −4.4]
B4 (blind)	+1.6 [+0.2, +3.0]	+2.8 [+0.8, +4.9]	+1.2 [−0.5, +3.1]
HEFT (static)	+5.2 [+4.2, +6.6]	−10.5 [−11.5, −9.5]	−14.9 [−16.2, −13.7]

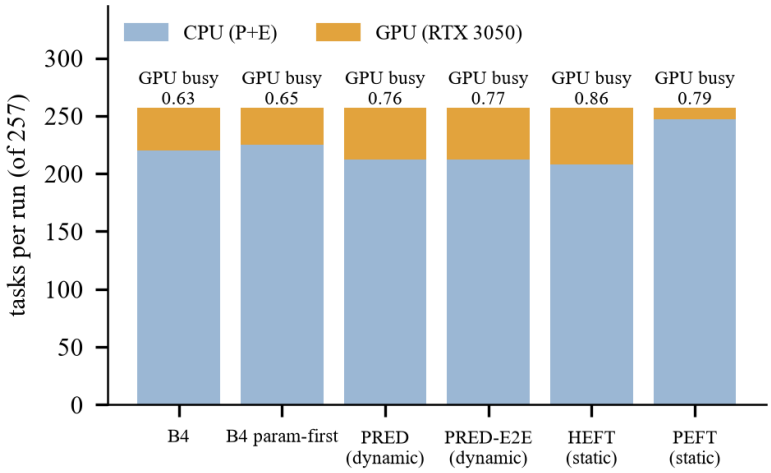


Figure 6. Tasks run on the CPU and on the GPU per policy (extended set, primary workload) and the GPU busy fraction.

Table 17. Static plans against HEFT_param_model on the primary workload (%, exploratory)

Hardware set	HEFT_E2E	PEFT	HEFT oracle (reference)
CPU only	+0.6 [−0.3, +1.6]	+17.7 [+16.8, +18.6]	−0.2 [−1.0, +0.5]
strict GPU	−4.1 [−5.5, −2.5]	+10.6 [+9.5, +11.5]	−3.1 [−4.3, −2.0]
extended GPU	+1.9 [+0.5, +3.3]	+18.5 [+16.9, +20.1]	+3.3 [+1.4, +5.5]

Adding the end-to-end terms to the static plan did not help consistently; PEFT is clearly the slowest static plan; the oracle plan, built from class-median costs, is a reference and not a bound. Planned over actual makespan of the static plans was between 0.88 and 1.03, yet a fixed assignment cannot react to the actual completion order.

Table 18. Primary workload, CPU only: old and end-to-end makespan (s) and the difference (ms)

Policy	Old convention	End-to-end	Difference (ms)
B4 (blind, dynamic)	3.998	4.010	12.5
B4 param-first (rule, dynamic)	3.297	3.310	12.8
PRED (dynamic)	3.268	3.297	29.2
PRED-E2E (dynamic)	3.288	3.318	30.8
HEFT (static)	3.437	3.478	40.7
HEFT-E2E (static)	3.457	3.499	42.4
PEFT (static)	4.012	4.093	81.3

Table 19. Primary workload, CPU only: initialisation, policy set-up, total scheduling time (ms) and mean queue wait (s)

Policy	Init.	Policy set-up	Scheduling	Queue wait
B4 (blind, dynamic)	7.7	2.6	6.7	0.571
B4 param-first (rule, dynamic)	8.0	2.6	6.6	0.560
PRED (dynamic)	7.7	19.0	16.0	0.624
PRED-E2E (dynamic)	7.8	20.5	22.8	0.626
HEFT (static)	7.6	30.4	29.1	0.613
HEFT-E2E (static)	7.6	32.2	29.4	0.615
PEFT (static)	7.6	71.1	31.0	0.554

7.3 Earlier study (exploratory) and the remote worker

On twenty chain orders never used before, the dispatcher was 19.3% (6P+7E) and 20.5% (2P+7E) faster than B4 and level with the hand rule; the gain was about 27% with few heavy chains and about 4% when most were heavy. There was no gain on Montage (every task prefers the P cores by 2.3 to 3.6 times), on a uniform batch or on a two-factor batch. In **emulation** the remote worker gave no benefit on a fast link although predicted and measured times matched (0.1457 s against 0.1464 s; the predictive policy was 3.78% [+2.27, +5.58] slower with it) and blind remote placement on a slow link cost 31%. These were not rerun with the final policies.

Table 20. Earlier exploratory sessions on unseen chain orders (effect against B4, negative = shorter)

Session	Blocks / orders	Dispatcher	Hand rule
6P+7E, 16 heavy	36 / 12	−19.32% [−20.0, −18.6]	−19.85%
2P+7E, 16 heavy	24 / 8	−20.45% [−21.2, −19.8]	−20.67%
6P+7E, 4 heavy	12 / 4	−27.65% [−34.0, −18.7]	−26.94%
6P+7E, 8 heavy	12 / 4	−26.76% [−27.9, −25.7]	−26.93%
6P+7E, 40 heavy	12 / 4	−3.81% [−5.1, −1.9]	−5.15%

Table 21. Defects and incidents found during the work

Item	Found by	Action
Static plans timed from the first dispatch, hiding planning time	design review	end-to-end makespan (amendment F1); old metric kept
Final-study stage 1 aborted: laptop unplugged, retry-id collision, low free memory (antivirus, Docker, WSL)	run logs	mains wait, first-unused-id logic; thresholds never lowered; attempts kept (_r2 to _r8)
Idle CUDA worker defeats the dispatch loop’s early exit	overhead diagnosis	quantified; fix in a separate adapter; not applied to the source
Learned model used on a profile it was not fitted on	audit	profile binding recorded in the model; refusal on mismatch
Remote node served one client then exited	loopback rehearsal	persistent mode and test

8. Existing versus Proposed Implementation

Table 22. Differentiation between the existing (base) idea and the proposed work

Aspect	Existing (B4 / base patent idea)	Proposed
Order of ready tasks	first-come first-served	predicted upward rank
Worker choice	first idle eligible worker	idle eligible worker with least predicted time; GPU only for verified stages
Cost knowledge	none	ridge model learned from logs; refitted by one command
Planning	none (dynamic)	compared against static HEFT/PEFT plans from the same model
Timing	dispatch to last completion	end-to-end, including start-up and planning
Primary workload	reference	−17.8% (CPU), −21.2% (strict GPU) vs B4; −5.2% and −12.2% vs static HEFT
No cost spread	reference	+0.0% (equivalent within ±2%)
Evidence	FPGA system (patent); simulation (papers)	real runs, paired intervals, ten Holm-adjusted contrasts, 521 tests

What prediction buys is ordering and informed placement; what it does not buy is anything beyond a rule that knows the cost parameter, anything where task cost does not vary, or an end-to-end benefit from accurate remote predictions alone. This agrees with [40] that order can matter more than estimate accuracy, now on real executions.

9. Testing, Reproducibility and Demonstration Guide

Every session stores design, environment and code hashes; the tables of the final report are generated from the stored analysis files by scripts; 520 of 521 tests pass. The pipeline is `scripts/final_stage1.ps1` (calibration, models, designs), `scripts/final_stage2.ps1` (six evaluation sessions), `scripts/final_analyze.py`, `scripts/final_results_table.py` and `scripts/final_build_report.py`. Commands assume the project root and the virtual environment (call `.\.venv\Scripts\python.exe` directly if PowerShell blocks activation).

```
.\.venv\Scripts\python.exe -m caosched.dashboard_v3 --port 8770 --events-dir
data\results\final_20261101\eval\sessions --open
.\.venv\Scripts\python.exe -m caosched.dashboard_v2 --port 8766 --events-dir
data\results\final_20261101\eval\sessions --open
.\.venv\Scripts\python.exe -m pytest -q
```

The dashboard shows the GPU (RTX 3050) as connected, the Raspberry Pi as *not connected*, the final results with hover details on screen 3, and, on screen 6 of v3, one live block of the final benchmark (about one to two minutes; needs 2.4 GB of free RAM). Five-minute narrative: a workflow is a task graph and a scheduler decides where and in what order; a static plan fixes this

beforehand, a dynamic policy decides while running; with one cost model the dynamic policy is 2% to 12% faster than the static plan and about 18% to 21% faster than blind dispatch when task cost varies; a rule that knows the cost parameter is almost as good; there is no gain without cost spread; the Raspberry Pi is not measured.

10. Publication and IPR Route

- **Primary candidate:** IEEE/ACM CCGrid 2027, full paper of 10 pages in IEEE two-column format, double-blind, abstract 24 Nov. 2026 and paper 1 Dec. 2026 as read from the call [41]; re-check the dates. Scopus indexing and quartile of every candidate must be verified on the publisher and Scopus pages before submission.
- **Alternatives:** a parallel/distributed-computing or future-generation-systems journal after adding real-link results; a workshop; a book chapter (no call verified).
- **IPR:** the base patent claims graph, queue and gating; rank-ordered learned dispatch is disclosed by [27]. The Invention Disclosure Form draft (document 08) must be filed *before* any public disclosure; a professional prior-art search is required. This is not legal advice.
- **Disclosure timing:** publishing the code or paper first can defeat novelty; the GitHub release is prepared but not pushed.

11. Conclusion, Limitations and Future Work

On a hybrid CPU+GPU laptop and one workload family, dynamic dispatch driven by a cheap cost model finishes a workflow 2% to 12% sooner than a static HEFT plan from the same model (5% on the CPU, 12% with the strict GPU set) and about 18% to 21% sooner than blind dispatch when task cost varies; it ties blind dispatch when it does not. A rule that knows the cost parameter reaches almost the same result, end-to-end terms and a hybrid selector add nothing, and the GPU helps only where a speed-up was measured and the model chooses the tasks.

Table 23. Claim status

Status	Claims
Supported	dynamic PRED beats blind B4 and the static HEFT plan from the same model (C1, C3, C5, C8); the GPU helps PRED (C9); end-to-end terms are equivalent (C2, C7)
Not established	advantage over a rule that knows the cost parameter; a hybrid selector; GPU benefit for blind B4; remote offloading benefit; any Raspberry Pi or real-network result; generality beyond this machine and workload family
Not supported / null	gain on Montage, on uniform and light workloads, on the two-factor workload; a remote worker helping

Status	Claims
	EFT at level 30

Limitations. One machine, one image-pipeline family at 512 px, one CUDA worker; only the primary session is confirmatory; static plans are occupancy-blind and the oracle plan uses class medians; features of the hybrid check were computed after the primary results were read; the source tree has uncommitted changes and sessions are never pooled across code hashes. **Future work:** a GPU cost model with lower error and an experiment that separates model error from decision timing; a second workload family; the remote worker with the final policies and a real link; a clean-tree rerun; a professional prior-art search.

References

[1] Raspberry Pi Ltd, “Raspberry Pi 5 product page,” https://www.raspberrypi.com/products/raspberry-pi-5/.

[2] Caltech/IPAC, “Montage: An astronomical image mosaic engine,” software, https://github.com/Caltech-IPAC/Montage (commit ac0a1533, used in this work).

[3] T. Coleman, H. Casanova, L. Pottier, M. Kaushik, E. Deelman, and R. Ferreira da Silva, “WfCommons: A framework for enabling scientific workflow research and development,” *Future Gener. Comput. Syst.*, vol. 128, pp. 16–27, 2022, doi: 10.1016/j.future.2021.09.043.

[4] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[5] H. Arabnejad and J. G. Barbosa, “List scheduling algorithm for heterogeneous systems by an optimistic cost table,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014, doi: 10.1109/TPDS.2013.57.

[6] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier, “StarPU: A unified platform for task scheduling on heterogeneous multicore architectures,” *Concurrency Comput.: Pract. Exper.*, 2011 (journal version; see also Euro-Par 2009, LNCS 5704, pp. 863–874).

[7] J. Luo, Y. Zhou, X. Li, M. Yuan, J. Zeng, and J. Yao, “Learning to optimize DAG scheduling in heterogeneous environment,” in *Proc. IEEE Int. Conf. Mobile Data Manage. (MDM)*, 2022, doi: 10.1109/MDM55031.2022.00040 (arXiv:2103.06980).

[8] X. Chen, J. Cao, Y. Sahni, S. Jiang, and Z. Liang, “Dynamic task offloading in edge computing based on dependency-aware reinforcement learning,” *IEEE Trans. Cloud Comput.*, vol. 12, no. 2, pp. 594–608, Apr.–Jun. 2024, doi: 10.1109/TCC.2024.3381646.

[9] Q. Tan, W. Chen, and D. Liu, “Task scheduling for heterogeneous multi-core processors based on deep reinforcement learning,” *Int. J. Intell. Syst.*, vol. 2025, Art. no. 7562400, 2025, doi: 10.1155/int/7562400.

[10] International Energy Agency, “Energy and AI,” Paris, France, Apr. 2025 (figures as reported in the summary and press coverage; the primary report page was not opened in this work).

[11] TOP500, “June 2025 list: highlights,” https://top500.org/lists/top500/2025/06/highs; accelerator counts and shares as analysed by The Next Platform, Jun. 10, 2025.

[12] IoT Analytics, “Number of connected IoT devices,” https://iot-analytics.com/number-connected-iot-devices/ (18.5 billion at end of 2024; 21.1 billion in 2025).

[13] Y. Cheng, Z. Chai, and A. Anwar, “Characterizing co-located datacenter workloads: An Alibaba case study,” arXiv:1808.02919, 2018.

[14] Intel, “What is Intel Thread Director?,” Intel support article 000097053, https://www.intel.com/content/www/us/en/support/articles/000097053/processors.html.

[15] Y. Song, C. Li, L. Tian, and H. Song, “A reinforcement learning based job scheduling algorithm for heterogeneous computing environment,” *Comput. Electr. Eng.*, vol. 107, Art. no. 108653, 2023, doi: 10.1016/j.compeleceng.2023.108653.

[16] C. Sun, T. Yang, and Y. Lei, “DDDQN-TS: A task scheduling and load balancing method based on optimized deep reinforcement learning in heterogeneous computing environment,” *Int. J. Intell. Syst.*, vol. 37, no. 11, pp. 9138–9172, 2022, doi: 10.1002/int.22983.

[17] J. Wang, J. Hu, G. Min, W. Zhan, A. Y. Zomaya, and N. Georgalas, “Dependent task offloading for edge computing based on deep reinforcement learning,” *IEEE Trans. Comput.*, 2022, doi: 10.1109/TC.2021.3131040.

[18] B. Gong and X. Jiang, “Dependent task-offloading strategy based on deep reinforcement learning in mobile edge computing,” *Wireless Commun. Mobile Comput.*, 2023, doi: 10.1155/2023/4665067.

[19] J. Liu, Y. Mi, X. Zhang, and X. Li, “Task graph offloading via deep reinforcement learning in mobile edge computing,” *Future Gener. Comput. Syst.*, vol. 158, pp. 545–555, 2024, doi: 10.1016/j.future.2024.04.034 (arXiv:2309.10569).

[20] Z. Cao, X. Deng, S. Yue, P. Jiang, J. Ren, and J. Gui, “Dependent task offloading in edge computing using GNN and deep reinforcement learning,” *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21632–21646, 2024, doi: 10.1109/JIOT.2024.3374969.

[21] G. Nieto, I. de la Iglesia, U. Lopez-Novoa, and C. Perfecto, “Deep reinforcement learning techniques for dynamic task offloading in the 5G edge-cloud continuum,” *J. Cloud Comput.*, vol. 13, Art. no. 94, 2024, doi: 10.1186/s13677-024-00658-0.

[22] J. Liu, P. Xie, W. Li, B. Tang, and J. Liu, “Dynamic task offloading scheme for edge computing via meta-reinforcement learning,” *Comput. Mater. Contin.*, vol. 82, no. 2, 2025, doi: 10.32604/cmc.2024.058810.

[23] C. Liu, H. Wang, M. Zhao, J. Liu, X. Zhao, and P. Yuan, “Dependency-aware online task offloading based on deep reinforcement learning for IoV,” *J. Cloud Comput.*, vol. 13, Art. no. 136, 2024, doi: 10.1186/s13677-024-00701-0.

[24] A. Rodríguez, A. Navarro, K. Nikov, J. Nunez-Yanez, R. Gran, D. Suárez Gracia, and R. Asenjo, “Lightweight asynchronous scheduling in heterogeneous reconfigurable systems,” *J. Syst. Archit.*, vol. 124, Art. no. 102398, 2022, doi: 10.1016/j.sysarc.2022.102398.

[25] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, “Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures,” *Future Gener. Comput. Syst.*, vol. 150, pp. 171–185, 2024, doi: 10.1016/j.future.2023.08.022.

[26] S. Nagpal, A. Karumannil, V. Jain, and A. K. Patil (Xilinx, Inc.), “Scheduling processing of machine learning tasks on heterogeneous compute circuits,” U.S. Patent 11,561,826 B1, Jan. 24, 2023 (filed Nov. 12, 2020).

[27] A. Vega *et al.* (International Business Machines Corp.), “Heterogeneous system on a chip scheduler with learning agent,” U.S. Patent 11,740,933 B2, Aug. 29, 2023.

[28] A. Amarnath *et al.* (International Business Machines Corp.), “Learning agent based application scheduling,” U.S. Patent 11,966,776 B2, Apr. 23, 2024.

[29] R. Barik, T. Shpeisman, B. T. Lewis, and R. Kaleem (Intel Corp.), “Adaptive scheduling for task assignment among heterogeneous processor cores,” U.S. Patent 10,186,007 B2, Jan. 22, 2019.

[30] R. Kim and M.-S. Kim (Samsung Electronics Co., Ltd.), “Method and apparatus for performing task scheduling in terminal,” U.S. Patent 10,162,671 B2, Dec. 25, 2018.

[31] Microsoft, “Quality of Service (Win32 apps),” Microsoft Learn, 2025, https://learn.microsoft.com/en-us/windows/win32/procthread/quality-of-service.

[32] The Linux Kernel documentation, “Energy Aware Scheduling,” https://docs.kernel.org/scheduler/sched-energy.html.

[33] The Kubernetes Authors, “Kubernetes Scheduler (kube-scheduler),” https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/.

[34] Amazon Web Services, “What is AWS Batch?,” AWS Batch User Guide, https://docs.aws.amazon.com/batch/latest/userguide/what-is-batch.html.

[35] SchedMD, “Generic Resource (GRES) scheduling,” Slurm Workload Manager documentation, https://slurm.schedmd.com/gres.html.

[36] Anyscale/Ray Project, “Scheduling — Ray Core documentation,” https://docs.ray.io/en/latest/ray-core/scheduling/index.html.

[37] Microsoft/Qualcomm, Copilot+ PC requirement of an NPU of at least 40 TOPS, as stated in vendor documents (e.g., Qualcomm, “What TOPS means: Snapdragon X Series”).

[38] T. Lu and S. Reda, “Agentic CPU-GPU scheduling for heterogeneous AI workloads,” arXiv:2607.22242, Jul. 2026.

[39] X. Wei *et al.*, “Agent.xpu: Efficient scheduling of agentic LLM workloads on heterogeneous SoC,” arXiv:2506.24045, 2025.

[40] A. Ilyushkin and D. Epema, “The impact of task runtime estimate accuracy on scheduling workloads of workflows,” in *Proc. IEEE/ACM Int. Symp. Cluster, Cloud Grid Comput. (CCGrid)*, 2018, pp. 331–341, doi: 10.1109/CCGRID.2018.00048.

[41] CCGrid 2027 Call for Papers, https://easychair.org/cfp/ccgrid27 (accessed Nov. 2026).

Appendix A. File Map

Table 24. Key locations (repository root D:\cao project)

What	Where
Source, tests, scripts	`src/caosched/`, `tests/`, `scripts/`
Final study data and evidence	`data/results/final_20261101/` (eval, calibration, models, preservation, ACTIVITY_LOG.md)
Final experiment report and table	`CAO_FINAL_EXPERIMENT_REPORT.md`, `RESULTS_TABLE_FINAL.csv`
Protocol and amendment	`docs/final/PROTOCOL_final.md`, `docs/final/AMENDMENT_F1_timing.md`
Dashboards	`src/caosched/dashboard/` (original), `dashboard_v2/`, `dashboard_v3/`
Submission package	`SUBMISSION_FINAL/` (this report, reviews, paper, deck, IPR documents, release bundle)